Universitatea Politehnica Timișoara

Facultatea de Automatică şi Calculatoare

Proiectarea cu Microprocesoare

Implementarea unui Microsistem cu Microprocesorul 8086

Autor: Ciobanu Daria-Andreea

Anul universitar 2025-2026

Tema Proiectului

Să se proiecteze un microsistem cu următoarea structură:

- unitate centrală cu microprocesorul 8086;
- 128 KB memorie EPROM, utilizând circuite 27C512;
- 64 KB memorie SRAM, utilizând circuite 62256;
- interfaţă serială, cu circuitul 8251, plasată în zona 04D0H – 04D2H sau 05D0H – 05D2H, în funcţie de poziţia microcomutatorului S1;
- interfaţă paralelă, cu circuitul 8255, plasată în zona 0250H – 0256H sau 0A50H – 0A56H, în funcţie de poziţia microcomutatorului S2;
- minitastatură cu 9 contacte;
- 10 led-uri;
- un modul de afişare cu 7 segmente, cu 8 ranguri (se pot afişa maxim 8 caractere hexa simultan);
- un modul LCD, cu 2 linii a câte 16 caractere fiecare, cu o interfaţă la alegerea studentului.

Toate programele în limbaj de asamblare vor fi concepute sub formă de subrutine. Programele necesare sunt:

- rutinele de programare ale circuitelor 8251 şi 8255;

- rutinele de emisie/ recepţie caracter pe interfaţa serială;

- rutina de emisie caracter pe interfaţă paralelă;

- rutina de scanare a minitastaturii;

- rutina de aprindere/ stingere a unui led;

- rutina de afişare a unui caracter hexa pe un rang cu segmente.

Structura rutinelor (intrări, secvenţe, ieşiri) va fi stabilită de fiecare student.

Descrierea Hardware-ului

Unitatea Centrală de Prelucrare (UCP sau CPU – Central Processing Unit) constituie nucleul sistemului de calcul proiectat, fiind utilizat pentru controlul si procesarea datelor. Arhitectura unității centrale din acest proiect este construită în jurul microprocesorului **Intel 8086** (un dispozitiv pe 16 biți) și include circuitele auxiliare necesare pentru generarea tactului și gestionarea magistralelor de date și adrese.

Schema bloc a unității centrale cuprinde:

- **Microprocesorul Intel 8086:** Elementul principal de procesare.
- **Generatorul de tact 8284A:** Asigură sincronizarea sistemului.
- **Logica de interfațare:** Circuite pentru demultiplexarea adreselor (**74LS373**) și pentru amplificarea/separarea magistralei de date (**74LS245**).

1. Microprocesorul Intel 8086

Intel 8086 este un microprocesor pe 16 biți, alimentat la o tensiune de +5V. Acesta dispune de o magistrală de date externă pe 16 biți și o magistrală de adrese pe 20 de biți, permițând adresarea directă a unui spațiu de memorie de 1 MB (2^{20} locații).

Structura internă a procesorului este divizată în două unități funcționale care operează în paralel:

- **BIU (Bus Interface Unit):** Gestionează interfața cu magistralele externe, aduce instrucțiunile și calculează adresele fizice.
- **EU (Execution Unit):** Decodifică și execută instrucțiunile preluate de BIU.

În cadrul acestui proiect, microprocesorul 8086 este configurat să funcționeze în **Modul Minim**, selectat prin conectarea pinului **MN/~MX** la nivel logic „1” (VCC). In acest mod microprocesorul gestionează el însuși semnalele de control, activarea memoriilor si dispozitivele de intrare/ieșire, plus ca un are nevoie de un bus extern.

Principalele semnale gestionate de procesor în această configurație sunt:

- **AD0-AD15 (Magistrala multiplexată):** Utilizează aceeași pini pentru date și adrese, reducând numărul de conexiuni necesare.
- **A15-A19:** Pini din rangurile de magistrala de adrese adiționale.
- **~BHE (Bus High Enable):** Indică un transfer de date pe octetul superior al magistralei (D8-D15).
- **ALE (Addres Latch Enable):** Activează memorarea adreselor din magistrala multiplexata.
- **~RD si ~WR :** Controlează operațiile de citire/intrare respectiv scriere/ieșire, sincronizând accesul la magistrale.
- **M/~IO (Memory/Input-Output):** Diferențiază accesul la memorie („1” logic) de accesul la porturile de I/O („0” logic).
- **DT/~R(Data Transmit/Receive):** Controlează direcția fluxului de date prin transceivere („1” pentru transmisie, „0” pentru recepție).

- **~DEN(Data Enable):** Validează transferul de date pe magistrala.
- **RESET:** Pentru resetarea/initializarea procesorului.
- **CLK:** Este intrarea semnalului de clock, in acest caz cu o frecventa data de dispozitivul 8284A.
- **READY:** Intrare de sincronizare cu memoriile sau perifericele lente; dacă este inactiv, procesorul introduce stări de așteptare (Wait States) pana cand perifericul sau memoria finalizeaza operatiunea.

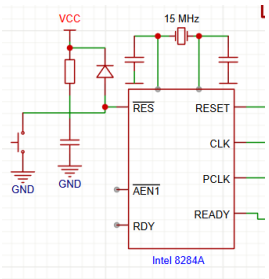
2. Generatorul de tact 8284A

Este esențial pentru funcționarea stabilă a sistemului, generând semnalul de ceas (CLK) pentru procesor și periferice, precum și semnalele de sincronizare RESET și READY. El primeste semnalul brut de la oscilatorul de cristal, care este conectat la el, si îl prelucrează oferind un semnal stabil pe care il va folosi microprocesorul.

Cristalul de cuarț de **15 MHz** este stabilizat prin două condensatoare, iar frecvența internă a procesorului va fi o treime din aceasta (5 MHz).

Ieșiri:

- **CLK:** Semnalul de tact principal pentru 8086.
- **PCLK:** Un semnal de frecvență redusă (jumătate din CLK) pentru periferice.
- **RESET:** Circuitul integrează logica de resetare (trigger Schmitt), fiind conectat la un circuit RC extern și un buton de reset, asigurând inițializarea corectă a sistemului la pornire (Power-On Reset).
- **READY**



3. Registrul de date 74x373

Magistrala de date și cea de adrese sunt multiplexate (partajează aceiași pini fizici AD0-AD15) si, de aceea, este necesară utilizarea unor circuite de memorare (latch-uri).

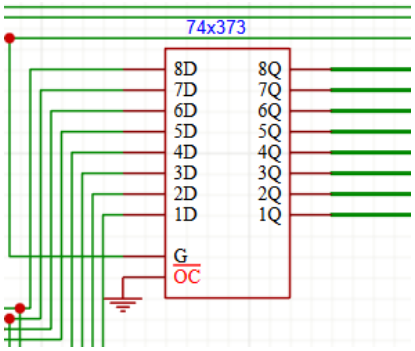
Astfel, este utilizat dispozitivul 72LS373, care este un registru de tip latch cu 8 bistabile D, având ieșiri cu trei stări), ceea ce permite conectarea mai multor dispozitive la aceeași magistrală fără interferențe. In schema noastră, sunt amplasate 3 astfel de registre pentru a memora adresele multiplexate de la microprocesor plus semnalul ~BHE.

Circuitul dispune de două semnale de control principale:

- **~OC (Output Control)** – controlează activarea ieșirilor; atunci când este „1” logic, ieșirile trec în a treia stare (izolate);

- **G (Latch Enable)** – controlează încărcarea bistabilelor, fiind activ la nivel logic „1”.

Astfel, în momentul activării semnalului G, datele de pe intrări sunt memorate și menținute la ieșiri până la următoarea activare, permițând procesorului să stabilească și să mențină stabilă adresa pe magistrală pe durata unui ciclu de instrucțiune. Pinii de validare G (Enable) ai latch-urilor sunt conectați la semnalul ALE generat de procesor. Când ALE este activ, adresa este capturată în latch-uri. Pinii ~OC sunt conectați la masă pentru a menține ieșirile active permanent.



/OC	G	8Q – 1Q
0	0	Vechiul conţinut
0	1	8D – 1D
1	X	A 3 – a stare

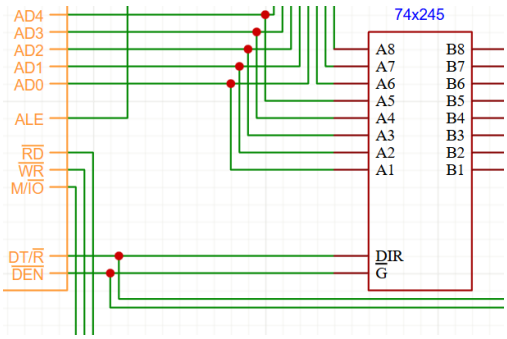
In primul registru stochează biți de la AD0 la AD7. Al doilea stochează biți de la AD8 la AD15. Si al treilea stochează biți de la A16 la A19 si semnalul ~BHE.

4. Circuitul amplificator 74x245

Circuitul 74x245 este utilizat pentru amplificarea și separarea magistralelor de date bidirecționale. El este altcatuit din opt perechi de porti bidirectionale cu trei stari, iar in proiect am folosit doua, pentru a gestiona transferul de date a 8086 si restul componentelor.

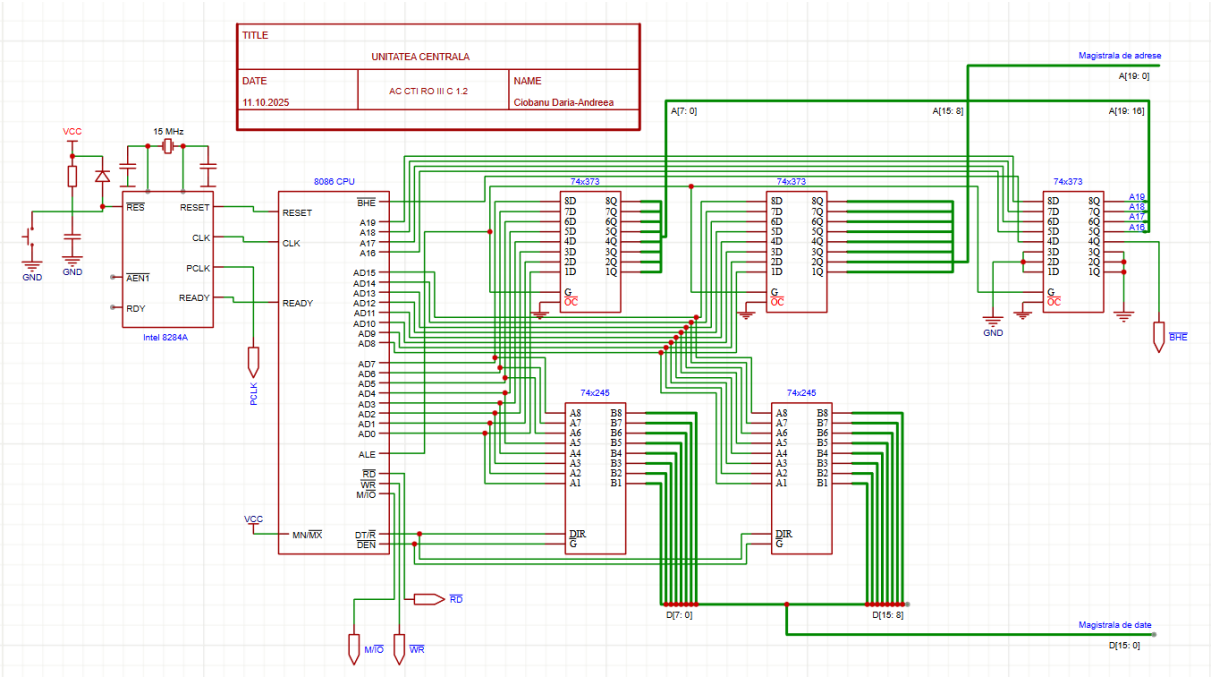
Controlul transferului este realizat astfel:

- **Pinul DIR (Direction):** Când este „1”, datele circulă de la procesor spre sistem (Scriere); când este „0”, datele vin dinspre sistem spre procesor (Citire). in proiect îl controlam cu semnalul DT/~R.
- **Pinul ~G (Enable):** La fel ca la 74x373, doar ca el este conectat la pinul ~DEN. Intrare de activare comună pentru toate liniile.



/G	DIR	A8 – A1	B8 – B1
0	0	B8 – B1	Intrări
0	1	Intrări	A8 – A1
1	X	A 3 – a stare	A 3 – a stare

Primul circuit 74x245 are intrările cu rangurile AD0 la AD7 din magistrala de date/adrese, si cu ieșirile D0-D7 din magistrala de adrese. Circuitul al doilea cuprinde rangurile de intrare AD8-AD15 si ieșirile la magistrala D8-D15.



Conectarea Memoriilor

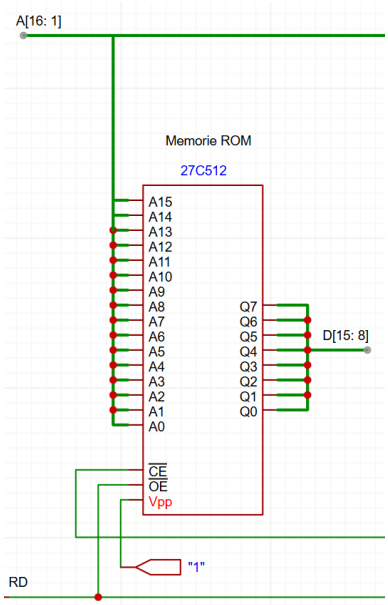
Proiectul utilizează doua tipuri de memorii:

- 128 KB de memorie EPROM, implementata cu doua circuite 27C512 (64 KB flecare)
- 64 KB de memorie SRAM, implementata cu doua circuite 62256 (32 KB fiecare)

Microprocesorul 8086 dispune de o magistrala de adresa pe 20 de biți, permițând accesarea unui spațiu maxim de 2^{20} = 1MB. Deci toate adresele posibile pentru acest procesor cuprind rangul: 00000H-FFFFFH. Când se construiește harta memoriei trebuie stabilite doua date pentru fiecare circuit (bloc) de memorie, adresa de început si limitele de adresare.

1. Pentru SRAM am ales sa incep de la adresa 00000H, iar limita superioara va fi calculata prin adaugarea capacitatii memoriei 64KB => 0FFFFH (2^{16} Bytes => 16 biti pentru codificare, deci primii 4 biti mai semnificativi vor fi folositi pentru decodificare)
2. Pentru EPROM am ales sa incep de la adresa E0000H, alocand urmatorii 128 KB pentru ea (E0000H + 1FFFFH = FFFFFH) . 2^{17} Bytes => 17 biti pentru codificare, deci primii 3 biti mai semnificativi vor fi folositi pentru decodificare.

1. Memoria EPROM 27C512



S-a utilizat o memorie de tip EPROM (Erasable Programmable Read-Only Memory), timp de acces tipic de 90-200 ns, pentru stocarea codului programului și a datelor constante, accesul fiind exclusiv de citire. Implementarea s-a realizat cu două circuite integrate **27C512** (fiecare având 64 KB, in total trebuia 128 KB).

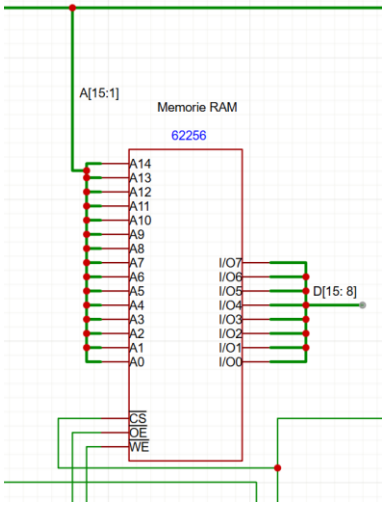
Memoria 27C512 este un dispozitiv nevolatil, care își păstrează conținutul în absența tensiunii de alimentare. Ștergerea datelor se poate realiza doar prin expunerea cipului de siliciu la lumină ultravioletă prin fereastra de cuarț a capsulei.

Cele 2 memorii folosite utilizeaza 16 biti de adresa – A[16:1] si sunt conectate una la bitii D[15:8], iar alta la D[7:0] de pe magistrala de date(linii bidirectionale).

Semnale de control:

- **~CE (Chip Enable):** Activează circuitul pentru acces la date si este conectat la o ieșire al decodificatorului de memorii.
- **~OE (Output Enable):** Activează ieșirile pentru citire si este conectat la pinul ~RD de la 8086.
- **Vpp:** După ce memoria a fost programata pinul VPP trebuie conectat la VCC(sau ii dam ,,1”) deoarece in aceasta configurație memoria va funcționa doar in modul citire, iar datele programate rămân intacte.

2. Memoria SRAM 62256



S-a utilizat o memorie de tip SRAM (*Static Random Access Memory*), timp de acces tipic 45-84 ns, pentru stocarea datelor temporare necesare procesării, având operații rapide de citire si scriere. Implementarea s-a realizat cu 2 circuite **62256** (fiecare avand 32 KB, in total trebuia sa avem 64KB).

Memoria 62256 este o memorie volatila care își pierde informația o data ce alimentarea este întrerupta si stocarea datelor se realizează prin bistabile de tip flip-flop care nu necesita de operați periodice pentru refresh.

Cele doua memorii utilizeaza 15 biti de adresa – A[15:1], iar una gestioneaza adresele pare – D[7:0], alta adresele impare – D[15:8].

Semnale de control:

- **~WE (Write Enable):** Spre deosebire de citire, scrierea în SRAM necesită o logică suplimentară pentru a preveni alterarea datelor din bancul greșit în timpul transferurilor pe 8 biți. Semnalul ~WE (Write Enable) pentru fiecare cip este generat prin compunerea semnalelor procesorului. **~WE_Par:** Activat de ~**WR** și **A0,** iar **~WE_Impar:** Activat de ~**WR** și ~**BHE.** Astfel, procesorul poate scrie independent în octetul inferior, în cel superior sau în ambii simultan.
- **~OE (Output Enable):** ~OE (Output Enable): Activează ieșirile pentru citire si este conectat la pinul ~RD de la 8086.
- **~CE (Chip Enable):** Activează circuitul pentru acces la date si este conectat la o ieșire al decodificatorului de memorii

3. Decodificare memoriilor

Pentru a genera semnalele de selecție corect pentru fiecare memorie in funcție de adresă, am implementat un decodificator bazat pe un circuit 74LS138 si logica combinațională. Se construiește un tabel care are ca si coloane rangurile magistralei de adrese iar ca rânduri prima si ultima configurație de adresa a fiecărei tip de memorie.

Memorie	A19	A18	A17	A16	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Adresa (Hex)
SRAM	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	00000h
	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0FFFFh
EPROM	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	E0000h
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	FFFFFh

Albastru: spatiul de adrese pentru SRAM

Portocaliu: spatiul de adrese pentru EPROM

Verde: Ranguri neschimbate, constante pe parcursul adreselor

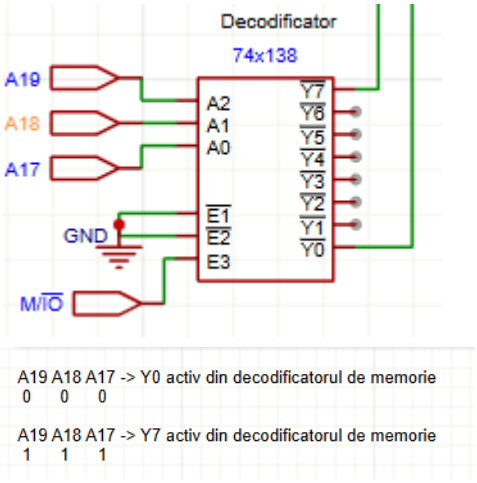
Decodificarea Completa presupune utilizarea tuturor biților de adresa relevanți pentru generarea semnalelor de selecție. Pornind de la rangul cel mai semnificativ spre cel mai puțin se rețin acele ranguri care rămân nemodificate pentru orice configurație de adresa din zona respectiva. In cazul proiectului, rangurile relevante pentru SRAM sunt A19, A18, A17,A16, iar pentru EPROM sunt A19, A18 si A17. Deci funcțiile combinaționale ar fi:

SEL_{SRAM} = ~A19 & ~A18 & ~A17 = ~y0 => Bitul A16 ar fi mereu 0 deci este irelevant daca s-ar mai face o poarta logica & cu el negat

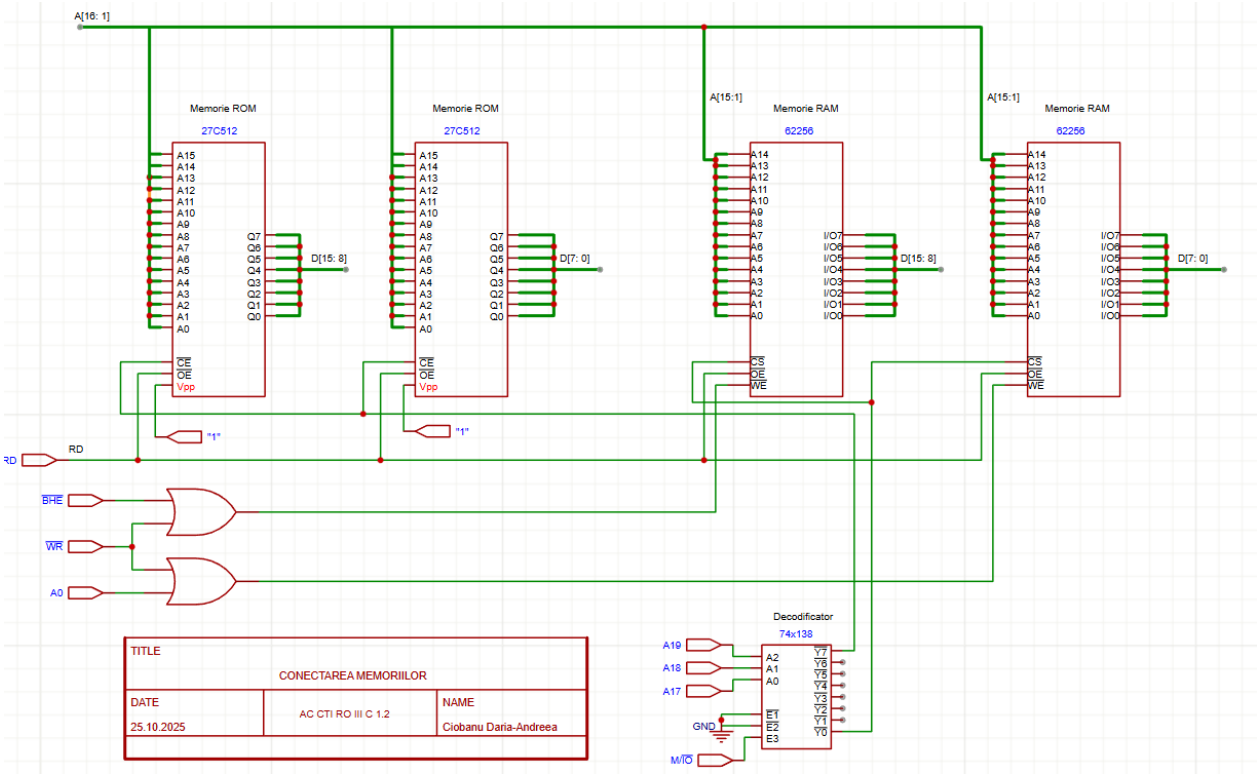
SEL_{EPROM}= A19 & A18 & A17 = ~y7

4. Circuitul decodificator 74LS138

In cazul asta, decodificatorul 74LS138 este configurat pentru a genera semnale de selecție pentru memoriile EPROM si SRAM. Când microprocesorul transmite o adresa pe magistrala, decodificatorul si logica combinațională determina memoria care trebuie activata.



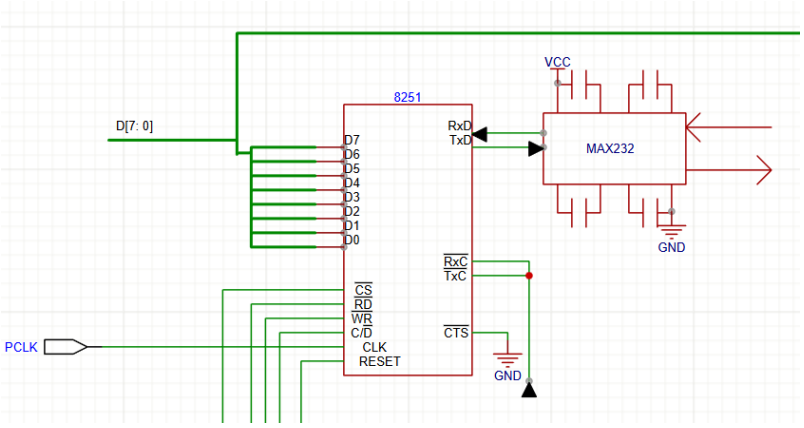
Legam la intrari A17, A18, A19 pentru selectie. Pinii ~E1 si ~E2 vor fi conectati la masă pentru ca decodificatorul va funcționa tot timpul timp ce E3 este conectat la M/~IO de la 8086 si va activa cu „1” logic funcționarea decodificatorului.



Interfață serială şi paralelă

1. Circuitul 8251 (interfață serială)

Pentru comunicația serială a sistemului am utilizat circuitul programabil **Intel 8251** (Universal Synchronous/Asynchronous Receiver/Transmitter). Acesta are rolul de a converti datele paralele primite de la procesor în flux serial pentru transmisie și invers pentru recepție, permițând comunicarea la distanță prin standardul RS-232.



Circuitul este programat de procesor prin două registre interne accesibile prin pinul **C/~D** (Control/Data). Inițializarea se face prin scrierea succesivă a două cuvinte de configurare:

- **Cuvântul de Mod (Mode Word):** Configurează formatul cadrului de date(biti de date, paritate, stop bit, sincronizare). **Valoare:** 1100 1110 => **0CEH**.
- **Cuvântul de Comandă (Command Word):** Controlează activarea transmisiei/recepției. Se activează Transmisia și Recepția. Se resetează flag-urile de eroare. **Valoare:** 0001 0101 =>**15H**.

Pini importanti:

- **C/~D (Comand/Data) :** selectează intre portul de date (0 logic) si portul de comanda/stări (1 logic), in cazul asta este legat la A1 de pe magistrala de adrese si se explica pentru ca avem asignata zona de adrese 04D0H -04D2H sau 05D0H - 05D2H, in funcție de poziția microcomutatorului S1
- **~WR si ~RD:** controlează operațiile de scriere si citire
- **~CS:** este conectat la ieșirea comutatorului S1 si activeaza sau dezactiveaza circuitul
- **RST:** legat la RESET
- **CLK:** legat la PCLK

Mod de functionare:

/CS	/RD	/WR	C//D	Operaţie
1	X	X	X	Magistrala de date în a 3-a stare
0	1	1	X	Magistrala de date în a 3-a stare
0	0	1	1	Citire a octetului de stare
0	0	1	0	Citire a datei
0	1	0	1	Scriere a cuvintelor de comandă
0	1	0	0	Scriere a datei

Deoarece 8251 operează la niveluri logice TTL (0-5V), iar standardul serial RS-232 necesită tensiuni bipolare (+- 12V), s-a utilizat circuitul de adaptare de nivel **MAX232**. Acesta inversează și amplifică semnalele de pe pinii **TxD** și **RxD**, utilizând o pompă de sarcină internă cu condensatoare externe.

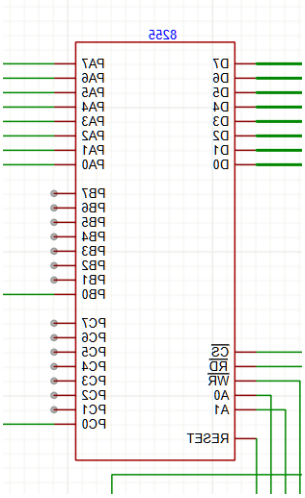
Pini de receptie:

- **RxD :** conectat la MAX232 si prin acesta se receptioneaza datele bit cu bit
- **~RxC:** este folosit in modul sincron pentru a sincroniza recepția datelor cu un semnal de clock

Pini de transmisie:

- **TxD:** transmite datele bit cu bit catre MAX232
- **~TxC :** la fel ca ~RXC doar ca pt transmitere

2. Circuitul 8255 (interfață paralelă)



Este utilizat pentru conectarea dispozitivelor periferice paralele, cum ar fi tastaturi, afișaje (cum este cazul cu un ecran LCD), sau alte module I/O. Permite comunicarea bidirecționala intre procesor si perifericele transmițând sau receptând date in mod paralel prin intermediul celor 3 porturi de cate 8 biți fiecare (Port A, Port B si Port C), unde fiecare poate funcționa ca intrare sau ieșire, deci are 24 de linii I/O.

Circuitul interfață programabilă 8255 oferă flexibilitate în comunicarea cu perifericele prin intermediul a trei regimuri distincte de operare:

- **Modul 0 (Intrare/Ieșire de bază):** Acesta reprezintă configurația utilizată în cadrul prezentului proiect. Modul 0 permite transferul simplu de date, fără

necesitatea unor semnale de sincronizare hardware (handshake) sau strobe. În această configurație, cele trei porturi pot fi definite independent ca intrări sau ieșiri, fiind ideal pentru interfațarea cu dispozitive care nu necesită protocoale complexe de comunicare.

- **Modul 1 (Intrare/Ieșire sincronizată):** Acest regim este destinat transferurilor de date care necesită validare (handshaking). În acest caz, Porturile A și B sunt utilizate pentru transferul propriu-zis de date, în timp ce liniile Portului C sunt alocate pentru generarea semnalelor de control și stare necesare sincronizării transferului.

- **Modul 2 (Transfer bidirecțional):** Acest mod permite comunicarea bidirecțională pe o singură magistrală de date și este disponibil exclusiv pentru Portul A. Similar cu modul anterior, liniile Portului C sunt utilizate pentru a gestiona direcția fluxului de date și pentru a asigura protocolul de control (handshake).

Se bazeaza pe urmatoarele conexiuni:

- **Semnale de control:** Pinii ~**WR** (scriere), ~**RD** (citire) și **RESET** au funcționalitate standard (similar circuitului 8251).
- **Selecția Cipului (~CS):** Activarea circuitului este controlată prin pinul ~CS, care este conectat fizic la microcomutatorul **S2** .

Mod de functionare:

/CS	/RD	/WR	A1	A0	Operaţia
0	1	0	0	0	Scriere în portul A
0	1	0	0	1	Scriere în portul B
0	1	0	1	0	Scriere în portul C
0	1	0	1	1	Scriere în portul cuvântului de comandă
0	0	1	0	0	Citire din portul A
0	0	1	0	1	Citire din portul B
0	0	1	1	0	Citire din portul C
0	0	1	1	1	Fără operaţie – magistrala de date este în a 3-a stare
0	1	1	x	x	Fără operaţie – magistrala de date este în a 3-a stare
1	x	x	x	x	Magistrala de date este în a 3-a stare

3. Decodificarea porturilor interfeței seriale si paralele (Circuitul 74LS138)

Se folosește decodificarea a porturilor pe interfețe pentru a putea selecta una dintre ele si in intervalul de adrese dat. Selectia se face pe baza magistralei de adrese știind intervalul de adrese dedicat pentru fiecare interfața si creând funcții logice care activează interfața corecta când microprocesorul adresează unul dintre aceste intervale. S-a utilizat un decodificator **74LS138** (3-la-8) pentru a plasa interfețele în spațiul de adrese de I/O.

	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Acces	--
04D0H	0	0	0	0	0	1	0	0	1	1	0	1	0	0	0	0	Date	Seriala 0 S1 = 0
04D2H	0	0	0	0	0	1	0	0	1	1	0	1	0	0	1	0	Comanda	
05D0H	0	0	0	0	0	1	0	1	1	1	0	1	0	0	0	0	Date	Seriala 1 S1 = 1
05D2H	0	0	0	0	0	1	0	1	1	1	0	1	0	0	1	0	Comanda	
0250H	0	0	0	0	0	0	1	0	0	1	0	1	0	0	0	0	Port A	Paralela 0 S2 = 0
0252H	0	0	0	0	0	0	1	0	0	1	0	1	0	0	1	0	Port B	
0254H	0	0	0	0	0	0	1	0	0	1	0	1	0	1	0	0	Port C	
0256H	0	0	0	0	0	0	1	0	0	1	0	1	0	1	1	0	Control	
0A50H	0	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	Port A	Paralela 1 S2 = 1
0A52H	0	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	Port B	
0A54H	0	0	0	0	1	0	1	0	0	1	0	1	0	1	0	0	Port C	
0A56H	0	0	0	0	1	0	1	0	0	1	0	1	0	1	1	0	Control	

Albastru : Adresele de porturi pentru Interfața Seriala cu comutatorul pe 0

Portocaliu: Adresele de porturi pentru Interfața Seriala cu comutatorul pe 1

Verde: Adresele de porturi pentru Interfața Paralela cu comutatorul pe 0

Roz: Adresele de porturi pentru Interfața Paralela cu comutatorul pe 1

Gri: Ranguri de selecție pentru decodificator

Functii combinationale pentru selectie:

Selectia pentru 8251 la S1: ~S1= ~A11 & ~A9 & ~A8 = ~Y0

S1= ~A11 & ~A9 & A8 = ~Y1

Selectia pentru 8255 la S2: ~S2= ~A11 & A9 & ~A8 = ~Y2

S2= A11 & A9 & ~A8 = ~Y6

A11	A10	A9	Output
0	0	0	~Y0 ; S1 = 0
0	0	1	~Y1 ; S1 = 1
0	1	0	~Y2 ; S2 = 0
0	1	1	Nefolosit
1	0	0	Nefolosit
1	0	1	Nefolosit
1	1	0	~Y6 ; S2 = 1
1	1	1	Nefolosit

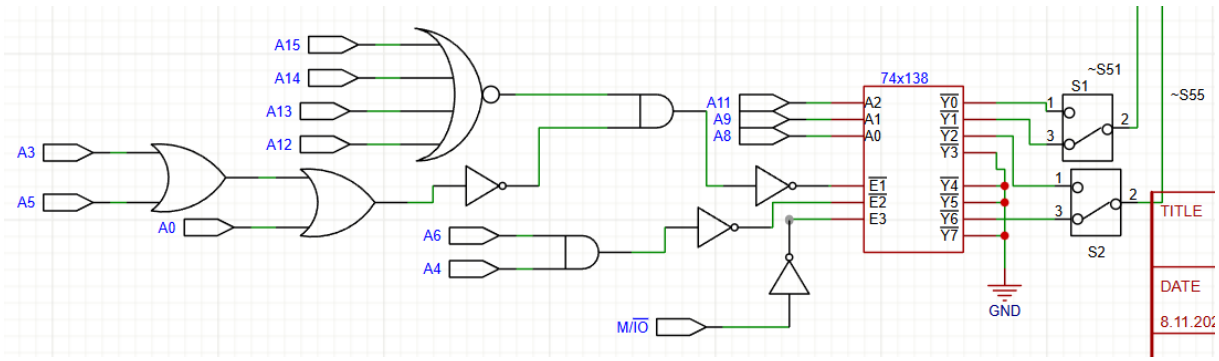
Pentru validarea funcționării decodificatorului, este necesară îndeplinirea condițiilor de **Enable**, determinate de biții de adresă cu valoare fixă. Mecanismul de selecție între memorie și periferice (I/O) se realizează prin semnalul de control **M/~IO** al procesorului 8086, conectat la pinul de validare **E3**:

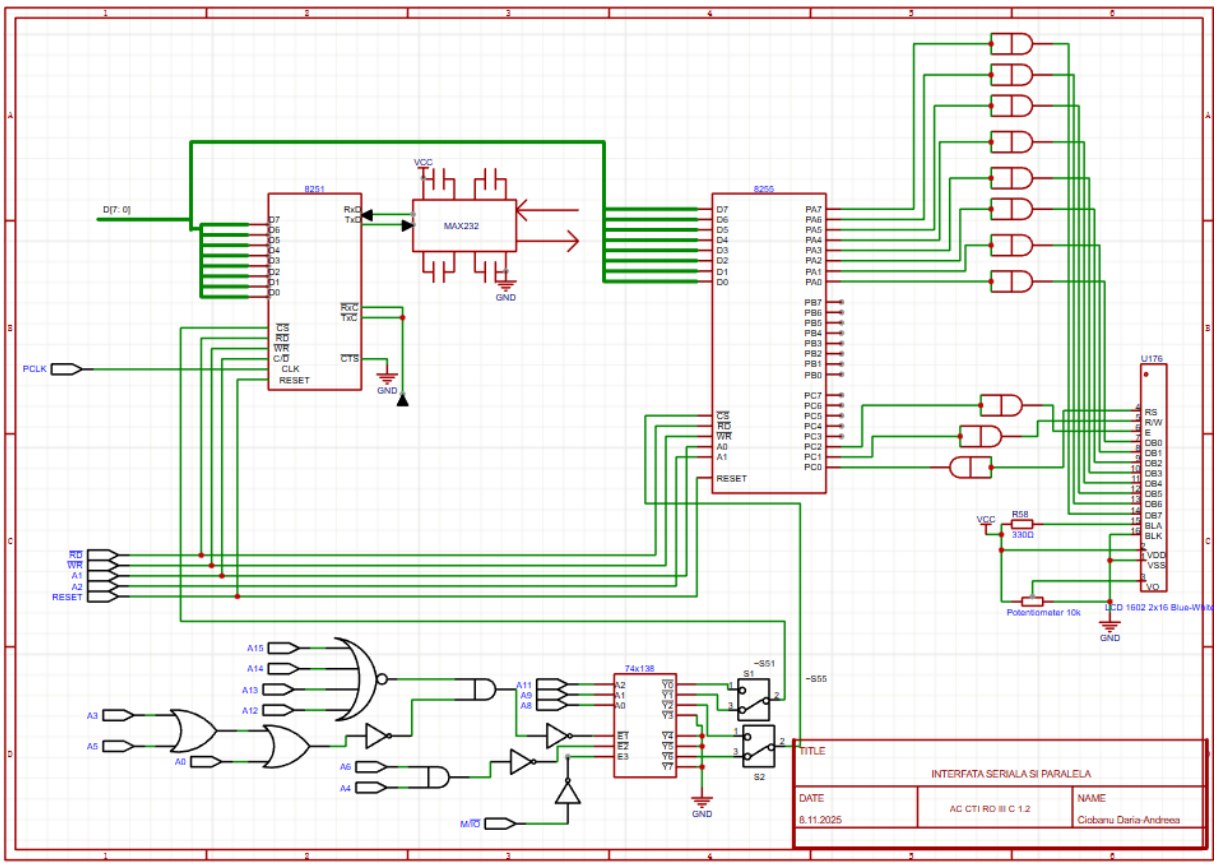
Grupam toate coloanele cu 0 intr-un enable:

E1= ~((~(A15 || A14 || A13 || A12)) && (~(A3 || A5 || A0)))

Grupam toate coloanele cu 1 intr-un enable:

E2= ~(A6 && A4)





Conectarea afișajelor

1. Minitastatura cu 9 contacte

Am integrat o minitastatură cu 9 butoane (switch-uri cu revenire), organizată sub forma unei matrici de **3 coloane și 3 linii**. Fiecare tastă este poziționată electric la intersecția dintre o linie și o coloană:

- **Coloanele:** Grupează tastele pe verticală (Col 1: 1-4-7, Col 2: 2-5-8, Col 3: 3-6-9). Acestea sunt conectate la anodul unor diode de protecție, iar catozii diodelor sunt legați la ieșirile registrului de date **74LS373** . Rolul diodelor este de a proteja portul de ieșire în cazul apăsării simultane a mai multor taste.

- **Liniile:** Grupează tastele pe orizontală (Linia 1: 1-2-3, Linia 2: 4-5-6, Linia 3: 7-8-9). Acestea sunt menținute la nivel logic ridicat (VCC) prin intermediul unor rezistențe de pull-up de 10 kOhm și sunt conectate la intrările circuitului tampon (buffer) **74LS244**.

În stare de repaus (*idle*), comutatoarele sunt deschise, iar intrările citite prin 74LS244 sunt pe „1 logic” datorită rezistențelor de pull-up.

Detectarea apăsării unei taste se realizează prin **algoritmul de scanare**:

1. **Activarea Coloanelor:** Software-ul comandă portul de ieșire (74LS373) pentru a pune, pe rând, câte o coloană în „0 logic” (L), lăsându-le pe celelalte pe „1”.

2. **Închiderea Circuitului:** La apăsarea unei taste, comutatorul scurtcircuitează linia cu coloana activată. Deoarece coloana este la masă (0V) prin registrul 74LS373, curentul va circula de la VCC prin rezistență și tastă către masă, trăgând linia respectivă în „0 logic”.

3. **Citirea și Decodificarea:** Procesorul citește starea liniilor prin portul de intrare (74LS244). Apariția unui „0 logic” pe o linie indică o apăsare.

4. **Identificarea:** Cunoscând coloana activată de software și linia citită ca fiind activă, se determină coordonatele unice ale tastei (intersecția linie-coloană), permițând identificarea tastei apăsate.

Circuitul 74LS244 are rolul de a amplifica curentul și de a separa magistrala de date a sistemului de mediul exterior, permițând citirea stării tastelor doar atunci când procesorul o solicită.

2. Conectarea a 10 LED-uri

Am aplasat 10 LED-uri conectate în configurație de **Anod Comun**. Pentru a emite lumină, LED-ul trebuie polarizat direct, asigurând o diferență de potențial între anod și catod care să depășească tensiunea de prag (Vt – specifică culorii LED-ului). Anodul este conectat la sursa de tensiune (VCC) prin intermediul unei rezistențe de **330 Ohm**. Aceasta are rolul de a limita curentul prin diodă și de a stabili punctul static de funcționare, determinând intensitatea luminoasă a LED-ului.

Magistrala de date este de 8 biți, deci pentru controlul celor 10 LED-uri este necesară utilizarea a două registre distincte.

Semnale de control sunt : ~OC (Output control), care este conectat permanent la masa(GND) pentru a mentine iesirile active si G(Enable) - pinul de validare a latch-ului este conectat la o ieșire dedicată din decodificatorul de adrese, permițând scrierea datelor doar atunci când procesorul adresează portul specific

3. Modul de afisare cu 7 segmente, cu 8 ranguri

Sistemul de afișare este compus din 8 module de 7 segmente, permițând afișarea a maxim 8 caractere simultan. Fiecare modul conține, din punct de vedere electric, 8 LED-uri independente (7 pentru segmentele cifrei și unul pentru punctul zecimal). Am optat pentru utilizarea afișajelor cu **Anod Comun.** Anodul comun al tuturor LED-urilor din modul este conectat la o tensiune pozitivă (VCC). Pentru a aprinde un segment, catodul corespunzător trebuie conectat la masă (GND). Astfel, comanda din procesor/registru se face în **logică negativă** (0 logic = segment aprins / 1 logic = segment stins).

Controlul celor 8 module se realizează prin **metoda nemultiplexată**. Aceasta presupune alocarea unui circuit de memorie dedicat pentru fiecare rang de afișare. Am utilizat 8 registre de tip latch **74x373.** Fiecare registru este adresat individual ca un port de ieșire. Când procesorul scrie date într-un registru (activând pinul G), acesta memorează configurația și menține segmentele aprinse continuu până la următoarea scriere.

Tabelul pentru intrarea la registrul de date 73x373:

HEX	DP	G	F	E	D	C	B	A	In hexa
0	1	1	0	0	0	0	0	0	C0h
1	1	1	1	1	1	0	0	1	F9h
2	1	0	1	0	0	1	0	0	A4h
3	1	0	1	1	0	0	0	0	B0h
4	1	0	0	1	`1	0	0	1	99h
5	1	0	0	1	0	0	1	0	92h
6	1	0	0	0	0	0	1	0	82h
7	1	1	1	1	1	0	0	0	F8h
8	1	0	0	0	0	0	0	0	80h
9	1	0	0	1	1	0	0	0	98h
A	1	0	0	0	1	0	0	0	88h
B	1	0	0	0	0	0	1	1	83h
C	1	1	0	0	0	1	1	0	C6h
D	1	0	1	0	0	0	0	1	A1h
E	1	0	0	0	0	1	1	0	86h
F	1	0	0	0	1	1	1	0	8Eh

4. Modul LCD, cu 2 linii a câte 16 caractere fiecare

Am utilizat un modul de afișare alfanumeric **LCD 1602**, capabil să afișeze 2 linii a câte 16 caractere. Deși controllerul LCD-ului permite funcționarea în modul de 4 biți (economisind pini prin transmiterea datelor în două jumătăți - nibbles), în acest proiect s-a ales modul de transfer pe 8 biți. Toți cei 8 pini de date ai LCD-ului (**DB0 - DB7**) sunt conectați la porturile A ai interfetei paralele, asigurând un transfer de date direct și o rutină software simplificată.

Gestionarea afișajului se face prin trei linii de comandă esențiale:

- **RS (Register Select):** Determină natura informației de pe magistrală.
    - 0 logic =>Se transmite o Comandă/Instrucțiune.
    - 1 logic => Se transmit Date (caractere ASCII).
- **R/W (Read/Write):** Setează direcția fluxului de date.
    - 0 logic => Scriere (Procesor spre LCD).
    - 1 logic => Citire (LCD spre Procesor - ex: citirea flag-ului de Busy).

- **E (Enable):** Semnalul de validare. Activarea efectivă a modulului și preluarea datelor se realizează pe tranziția acestui semnal (impuls 0 => 1).

5. Decodificarea porturilor pentru afișaje (Circuitul 74LS154)

Pentru a gestiona accesul exclusiv la multiplele periferice de ieșire și intrare (LED-uri, afișaje cu 7 segmente, tastatură), este necesar un mecanism de decodificare a adreselor. Deoarece sistemul necesită controlul a 12 porturi distincte (2 pentru LED-uri, 8 pentru segmente și 2 pentru minitastatură), s-a optat pentru utilizarea unui decodificator 4-la-16 (U44), tipic un 74LS154. Din cele 16 ieșiri disponibile, sunt utilizate efectiv 12.

Alocarea adreselor s-a realizat în intervalul **0300H – 032CH**. Adresele sunt distanțate cu un pas de 4 octeți (offset = 4). Această distanțare permite utilizarea liniilor de adresă **A2, A3, A4 și A5** direct la intrările de selecție ale decodificatorului. Biții A0 și A1 sunt ignorați în procesul de selecție a cipului (*Chip Select*), ceea ce caracterizează metoda ca fiind o **decodificare incompletă** (partial decoding).

Adresa	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Porturi, afisaje, tastatura
0300h	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	SL1
0304h	0	0	0	0	0	0	1	1	0	0	0	0	0	1	0	0	SL2
0308h	0	0	0	0	0	0	1	1	0	0	0	0	1	0	0	0	SA1
030Ch	0	0	0	0	0	0	1	1	0	0	0	0	1	1	0	0	SA2
0310h	0	0	0	0	0	0	1	1	0	0	0	1	0	0	0	0	SA3
0314h	0	0	0	0	0	0	1	1	0	0	0	1	0	1	0	0	SA4
0318h	0	0	0	0	0	0	1	1	0	0	0	1	1	0	0	0	SA5
031Ch	0	0	0	0	0	0	1	1	0	0	0	1	1	1	0	0	SA6
0320h	0	0	0	0	0	0	1	1	0	0	1	0	0	0	0	0	SA7
0324h	0	0	0	0	0	0	1	1	0	0	1	0	0	1	0	0	SA8
0328h	0	0	0	0	0	0	1	1	0	0	1	0	1	0	0	0	ST1
032Ch	0	0	0	0	0	0	1	1	0	0	1	0	1	1	0	0	~ST2

Albastru: Adresele de porturi pentru Leduri.

Verde: Adresele de porturi pentru afisaj cu 8 segmente

Roz: Adresele de porturi pentru minitastatura

Gri: Ranguri de selecție pentru decodificator

Pentru tabelul de adevar de la iesirea decodificatorului se face conversia din binar in zecimal. De exemplu: 0000 => ~Y0

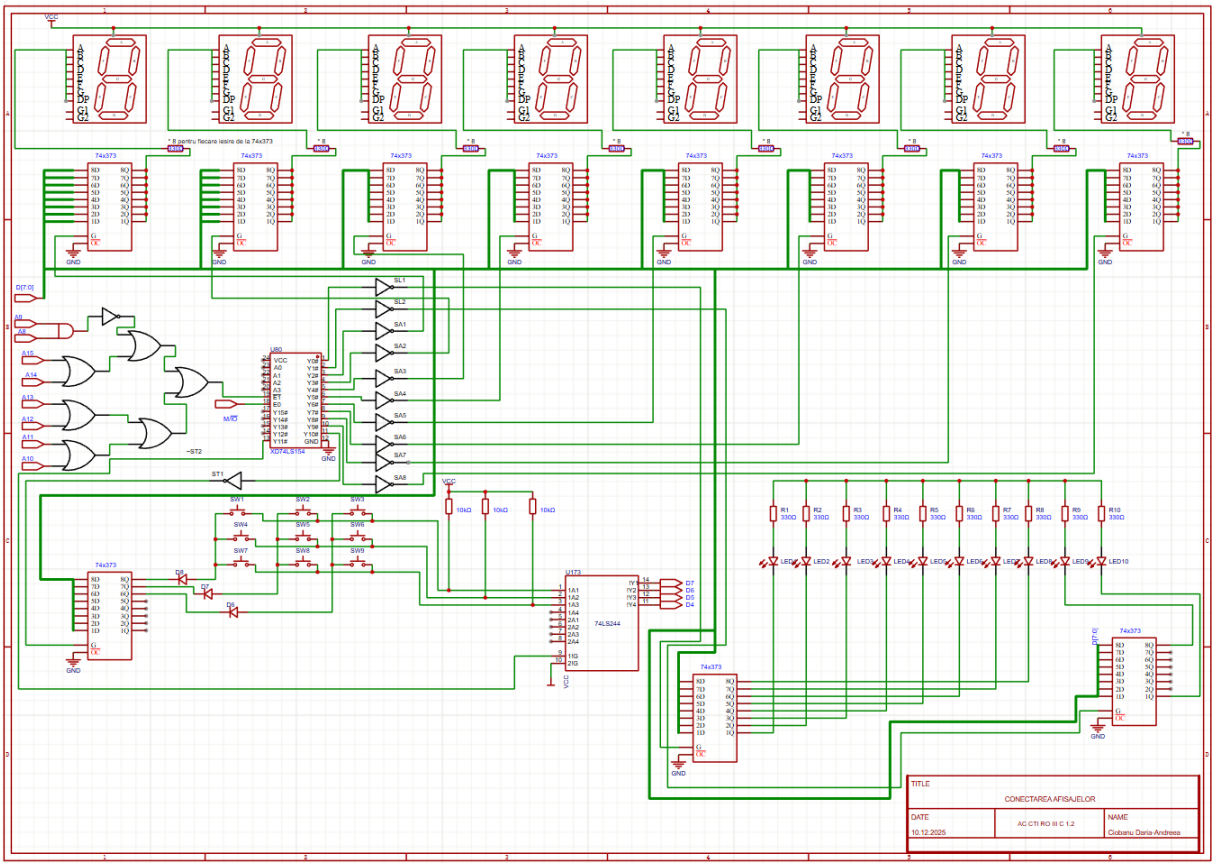
Există o discrepanță între logica de ieșire a decodificatorului și logica de activare a perifericelor:

- **Decodificatorul:** Are ieșiri active pe **„0 logic”** (Active Low).
- **Registrele periferice (ex. 74LS373):** Au pinii de validare (Enable/G) activi pe **„1 logic”**.

Pentru a asigura compatibilitatea, se intercalează **inversoare (porți NOT)** pe liniile de semnal dintre decodificator și registrele de ieșire

La fel ca la decodificatorul de porturi pentru interfețe, avem nevoie de semnale enable pentru activarea sau dezactivarea decodificatorului. Trebuie alese rangurile altfel încât adresele de la porturile de la afișaje si minitastatura sa nu interfereze cu adresele pentru porturile de la interfețe. Pentru ~E1 se folosește semnalul M/~IO de la 8086 (ca la decodificatorul pentru interfețe)

~E1 = A15 | A14 | A13 | A12 | A11 | A10 | (A9 & A8)



Descrierea Software

1. Rutinele de programare ale circuitelor 8251 – interfata seriala

```
; Definire adrese porturi COMANDA/STARI (C/D = 1)
CMD_PORT_S1_0  EQU 04D2H
CMD_PORT_S1_1  EQU 05D2H
; Cuvant de mod
MODE_WORD      EQU 0CEH    ; 11001110b (Asincron, 8 biti, x16, 2 Stop)
COMMAND_WORD   EQU 15H     ; 00010101b (TxEN, RxEN, ErrReset)
INIT_8251 PROC
   ; Input: AL contine pozitia S1 (0 sau 1)
   ; Output: Circuitul 8251 este initializat
   ; 1. Selectia adresei de baza in functie de S1
   CMP AL, 0            ; Verificam daca S1 = 0
   JNE SET_ADDR_S1_1      ; Daca nu, sarim la adresa pentru S1=1

   MOV DX, CMD_PORT_S1_0       ; DX = 04D2H
   JMP SEND_INIT

SET_ADDR_S1_1:
   MOV DX, CMD_PORT_S1_1  ; DX = 05D2H

SEND_INIT:
   ; 2. Trimiterea Cuvantului de MOD
   MOV AL, MODE_WORD      ; Incarcam 0CEH
   OUT DX, AL           ; Scriem la portul de comand
   ; 3. Trimiterea Cuvantului de COMANDA
   MOV AL, COMMAND_WORD   ; Incarcam 15H
   OUT DX, AL           ; Scriem la portul de comanda

   RET
INIT_8251 ENDP
```

2. Rutinele de programare ale circuitelor 8255 – interfata paralela

```
; Adresele Registrului de Control (Offset +6 fata de baza)
CMD_PORT_8255_S2_0  EQU 0256H   ; Pentru S2 = 0
CMD_PORT_8255_S2_1  EQU 0A56H   ; Pentru S2 = 1
; Cuvantul de Control: Mod 0, Toate porturile IESIRE (A, B, C)
; 1000 0000 B = 80H  (caz 1 pt mod 0 de pe curs 8)
PPI_CTRL_WORD       EQU 80H

INIT_8255 PROC
   PUSH DX            ; Salvam DX pentru a nu altera restul programului
   ; 1. Selectia adresei de baza in functie de S2 (AL)
   CMP AL, 0          ; Verificam daca S2 = 0
   JNE ADDR_S2_1      ; Daca nu, sarim la adresa pentru S2=1

   ; Cazul S2 = 0
   MOV DX, CMD_PORT_8255_S2_0 ; Incarcam adresa 0256H
   JMP WRITE_CFG

ADDR_S2_1:
   ; Cazul S2 = 1
   MOV DX, CMD_PORT_8255_S2_1 ; Incarcam adresa 0A56H

WRITE_CFG:
   ; 2. Trimiterea Cuvantului de Control
   MOV AL, PPI_CTRL_WORD   ; Incarcam 80H
   OUT DX, AL             ; Scriem in registrul de control al 8255

   ; La 8255 nu este nevoie strict de delay ca la 8251,
   ; dar configurarea este instanta.

   POP DX             ; Refacem registrul DX
   RET
INIT_8255 ENDP
```

3. Rutinele de emisie/ recepţie caracter pe interfaţa serială

```
; Adrese pentru S1 = 0 (Pozitia 1 a comutatorului)
PORT_8251_1_DATA   EQU 04D0H      ; Adresa de DATE (A1=0)
PORT_8251_1_CTRL   EQU 04D2H      ; Adresa de STATUS/COMANDA (A1=1)
; Adrese pentru S1 = 1 (Pozitia 2 a comutatorului)
PORT_8251_2_DATA   EQU 05D0H      ; Adresa de DATE (A1=0)
PORT_8251_2_CTRL   EQU 05D2H      ; Adresa de STATUS/COMANDA (A1=1)
; Masti pentru Registrul de Stare (Conform Curs 8, Pagina 30)
MASK_TXRDY        EQU 01H        ; Bit 0 = TxRD (Transmitter Ready)
MASK_RXRDY        EQU 02H        ; Bit 1 = RxRD (Receiver Ready)
;==============================================================================
;  RUTINA EMISIE CARACTER ; Descriere: Trimite un caracter pe seriala
; Input:  CL = Caracterul de transmis;  AL = Pozitia S1 (0 sau 1)
TRANSMITE_CARACTER_SERIALA PROC
   PUSH DX           ; Salvam DX
   PUSH AX           ; Salvam AX

   ; Pas 1: Selectam portul de STATUS in functie de S1
   CMP AL, 0
   JNE SEL_TX_S1_1     ; Daca AL != 0, mergem la setarile pentru S1=1

   ; Cazul S1 = 0
   MOV DX, PORT_8251_1_CTRL
   JMP WAIT_TX

SEL_TX_S1_1:
   ; Cazul S1 = 1
   MOV DX, PORT_8251_2_CTRL

WAIT_TX:
   ; Pas 2: Citim registrul de stare
   IN AL, DX          ; Citim Statusul
   TEST AL, MASK_TXRDY ; Verificam Bit 0 (TxRDY)
   JZ WAIT_TX         ; Daca e 0, transmitatorul e ocupat, asteptam
```

```
    ; Pas 3: Calculam adresa de DATE
    ; Scadem 2 din adresa de Control (ex: 04D2H - 2 = 04D0H)
    SUB DX, 2

    ; Pas 4: Trimitem caracterul
    MOV AL, CL          ; Mutam caracterul in AL
    OUT DX, AL          ; Scriem la portul de DATE

    POP AX              ; Refacem registrele
    POP DX
    RET
TRANSMITE_CARACTER_SERIALA ENDP
;==============================================================================
; 3. RUTINA RECEPTIE CARACTER
; Descriere: Asteapta si citeste un caracter de la seriala; Input:  AL = Pozitia S1 (0 sau 1)
; Output: AL = Caracterul receptionat
RECEPTIE_CARACTER_SERIALA PROC
    PUSH DX             ; Salvam DX (dar nu si AX, in el intoarcem rezultatul)

    ; Pas 1: Selectam portul de STATUS in functie de S1
    CMP AL, 0
    JNE SEL_RX_S1_1

    ; Cazul S1 = 0
    MOV DX, PORT_8251_1_CTRL
    JMP WAIT_RX

SEL_RX_S1_1:
    ; Cazul S1 = 1
    MOV DX, PORT_8251_2_CTRL

WAIT_RX:
    ; Pas 2: Citim registrul de stare
```

```
   IN AL, DX          ; Citim Statusul
   TEST AL, MASK_RXRDY ; Verificam Bit 1 (RxRD)
   JZ WAIT_RX         ; Daca e 0, nu am primit nimic, asteptam

   ; Pas 3: Calculam adresa de DATE
   SUB DX, 2          ; Trecem de la CTRL (xxD2) la DATA (xxD0)

   ; Pas 4: Citim caracterul
   IN AL, DX          ; Citim octetul primit de la portul de DATE

   POP DX             ; Refacem DX
   RET
RECEPTIE_CARACTER_SERIALA ENDP
```

4. Rutinele de emisie caracter pe interfaţa paralelă

```
; Descriere: Trimite un caracter catre LCD 1602; Input:  CL = Caracterul
;        AL = Pozitia S2 (0 sau 1) ; Hardware: Port A = Date (DB0-DB7);        Port C = Control (PC0=RS, PC1=RW, PC2=E)
;==============================================================================
; Constante pentru Control LCD (pe Port C)
LCD_RS      EQU 01H     ; PC0 = 1 (Selectie Data)
LCD_RW      EQU 00H     ; PC1 = 0 (Scriere)
LCD_E       EQU 04H     ; PC2 = 1 (Enable High)

EMISIE_LCD PROC
   PUSH DX
   PUSH AX
   PUSH BX            ; Salvam si BX

   ; 1. Selectia Bazei in functie de S2
   CMP AL, 0
   JNE ADDR_LCD_S2_1

   ; S2 = 0 -> Baza 0250H
   MOV BX, 0250H       ; BX = Adresa Port A (Date)
   JMP SEND_LCD
```

```
ADDR_LCD_S2_1:
    ; S2 = 1 -> Baza 0A50H
    MOV BX, 0A50H       ; BX = Adresa Port A (Date)

SEND_LCD:
    ; 2. Scriem DATA pe Portul A
    MOV DX, BX          ; DX = Port A
    MOV AL, CL          ; Caracterul de afisat
    OUT DX, AL          ; Scriem datele pe magistrala

    ; 3. Generam semnalele de control pe Portul C (RS=1, RW=0, E=Impuls)
    ADD DX, 4           ; DX devine Port C (0254H sau 0A54H)

    ; Faza A: Setam RS=1, RW=0, E=1 (Enable Activ)
    MOV AL, LCD_RS OR LCD_E  ; 0000 0101 (PC2=1, PC0=1)
    OUT DX, AL

    ; Faza B: Mic Delay (LCD-ul are nevoie de timp sa "prinda" datele)
    NOP
    NOP

    ; Faza C: Setam E=0 (Front descrescator -> LCD preia datele)
    MOV AL, LCD_RS          ; 0000 0001 (PC2=0, PC0=1)
    OUT DX, AL

    POP BX
    POP AX
    POP DX
    RET
EMISIE_LCD ENDP
```

5. Rutina scanare minitastatura

```
KEY_OUT_PORT    EQU 0328H       ; Port Iesire (Coloane)
KEY_IN_PORT     EQU 032CH       ; Port Intrare (Linii)
; Masti pentru activarea coloanelor (Active Low)
COL_1_ACTIVE    EQU 0FEH        ; 1111 1110 (Col 1 = 0)
COL_2_ACTIVE    EQU 0FDH        ; 1111 1101 (Col 2 = 0)
COL_3_ACTIVE    EQU 0FBH        ; 1111 1011 (Col 3 = 0)
; Masti pentru citirea liniilor (Active Low)
ROW_1_MASK      EQU 01H         ; Bit 0
ROW_2_MASK      EQU 02H         ; Bit 1
ROW_3_MASK      EQU 04H         ; Bit 2
;==================================================================================
; RUTINA PRINCIPALA DE SCANARE
;==================================================================================
SCAN_KEYPAD PROC

REIA_SCANARE:
   ; --- PAS 1: Activam COLOANA 1 (Tastele 1, 4, 7) ---
   MOV DX, KEY_OUT_PORT
   MOV AL, COL_1_ACTIVE    ; Punem 0 pe Col 1
   OUT DX, AL

   ; Citim Liniile
   MOV DX, KEY_IN_PORT
   IN AL, DX

   ; Verificam Linia 1 (Tasta 1)
   TEST AL, ROW_1_MASK     ; Bit 0 e 0?
   JZ TASTA_1_PRESSED

   ; Verificam Linia 2 (Tasta 4)
   TEST AL, ROW_2_MASK     ; Bit 1 e 0?
   JZ TASTA_4_PRESSED
```

```
; Verificam Linia 3 (Tasta 7)
TEST AL, ROW_3_MASK     ; Bit 2 e 0?
JZ TASTA_7_PRESSED

; --- PAS 2: Activam COLOANA 2 (Tastele 2, 5, 8) ---
MOV DX, KEY_OUT_PORT
MOV AL, COL_2_ACTIVE    ; Punem 0 pe Col 2
OUT DX, AL

MOV DX, KEY_IN_PORT
IN AL, DX

TEST AL, ROW_1_MASK     ; Bit 0 e 0?
JZ TASTA_2_PRESSED

TEST AL, ROW_2_MASK     ; Bit 1 e 0?
JZ TASTA_5_PRESSED

TEST AL, ROW_3_MASK     ; Bit 2 e 0?
JZ TASTA_8_PRESSED

; --- PAS 3: Activam COLOANA 3 (Tastele 3, 6, 9) ---
MOV DX, KEY_OUT_PORT
MOV AL, COL_3_ACTIVE    ; Punem 0 pe Col 3
OUT DX, AL

MOV DX, KEY_IN_PORT
IN AL, DX

TEST AL, ROW_1_MASK     ; Bit 0 e 0?
JZ TASTA_3_PRESSED

TEST AL, ROW_2_MASK     ; Bit 1 e 0?
JZ TASTA_6_PRESSED
```

```
    TEST AL, ROW_3_MASK     ; Bit 2 e 0?
    JZ TASTA_9_PRESSED

    ; Daca nu s-a apasat nimic, reluam ciclul
    JMP REIA_SCANARE

SCAN_KEYPAD ENDP

; SUBRUTINE TRATARE TASTE (Exemplu pentru Tasta 1)
TASTA_1_PRESSED:
    CALL DELAY_DEBOUNCE     ; Asteptam stabilizarea contactului mecanica

    ; Verificam daca tasta e inca apasata (pentru a elimina zgomotul)
    MOV DX, KEY_IN_PORT
WAIT_RELEASE_1:
    IN AL, DX
    TEST AL, ROW_1_MASK
    JZ WAIT_RELEASE_1       ; Asteptam cat timp tasta este apasata (0)

    CALL DELAY_DEBOUNCE     ; Delay dupa eliberare

    ; Aici punem codul specific tastei 1 (ex: Afisare pe LCD, aprindere LED)
    ; MOV CL, '1'
    ; CALL EMISIE_CARACTER_LCD
    ; ACTIUNE: Afisam '1' pe LCD
    MOV CL, '1' ; Caracterul MOV AL, 0
   ; Selectam S2=0 (Baza 0250H)
   CALL EMISIE_LCD
   ; Apelam rutina de afisare
    JMP REIA_SCANARE        ; Revenim la scanare

; (La fel trebuie facut pentru TASTA_2... TASTA_9)
; RUTINA INTARZIERE (Debounce)
```

```
DELAY_DEBOUNCE PROC
   PUSH CX
   MOV CX, 2000H          ; Valoare ajustabila (~10-20ms)
LOOP_D:
   LOOP LOOP_D
   POP CX
   RET
DELAY_DEBOUNCE ENDP

CODE ENDS
```

6. Rutina de aprindere/stingere a unui led

```
; Adresele porturilor din proiectul tau (Decodificator U44)
LED_PORT_1  EQU 0300H      ; Controleaza LED 0 - 7
LED_PORT_2  EQU 0304H      ; Controleaza LED 8 - 9

; Tabel pentru activarea unui singur LED (Logica 0 = ON)
; Pozitia bitului de 0 indica LED-ul aprins
LED_MASKS   DB 0FEH, 0FDH, 0FBH, 0F7H, 0EFH, 0DFH, 0BFH, 07FH     ; Bit0, Bit1, Bit2, Bit3, Bit4, Bit5, Bit6, Bit7

; RUTINA: APRINDE_LED ; Input:  AL = Numarul LED-ului (0 - 9) ; Hardware: Anod Comun (0=ON, 1=OFF)
APRINDE_LED PROC
   PUSH BX
   PUSH DX

   ; Verificam daca LED-ul e pe Portul 1 (0-7) sau Portul 2 (8-9)
   CMP AL, 8
   JAE LED_PORT_2_ON   ; Daca AL >= 8, mergem la Port 2

   ; --- Cazul Port 1 (LED 0-7) ---
   LEA BX, LED_MASKS   ; Incarcam adresa tabelului
   XLAT                ; AL = LED_MASKS[AL] (Extrage masca corecta)
   MOV DX, LED_PORT_1  ; Adresa 0300H
   OUT DX, AL          ; Trimitem comanda (Bitul 0 aprinde LED-ul)
```

```
    JMP EXIT_ON

LED_PORT_2_ON:
    ; --- Cazul Port 2 (LED 8-9) ---
    SUB AL, 8          ; Ajustam indexul (8 devine 0, 9 devine 1)
    LEA BX, LED_MASKS   ; Incarcam adresa tabelului
    XLAT               ; Extrage masca (ex: FEH pentru LED 8)
    MOV DX, LED_PORT_2  ; Adresa 0304H
    OUT DX, AL         ; Trimitem comanda

EXIT_ON:
    POP DX
    POP BX
    RET
APRINDE_LED ENDP

; RUTINA: STINGE_TOT (Sau un LED specific prin suprascriere); Descriere: Stinge toate LED-urile de pe portul selectat
; Input: AL = Numarul LED-ului (pentru a selecta portul corect)
STINGE_LED PROC
    PUSH DX
    CMP AL, 8
    JAE LED_PORT_2_OFF
    ; --- Stinge tot pe Port 1 ---
    MOV DX, LED_PORT_1
    MOV AL, 0FFH       ; 1111 1111 -> Toate LED-urile OFF
    OUT DX, AL
    JMP EXIT_OFF
LED_PORT_2_OFF:
    ; --- Stinge tot pe Port 2 ---
    MOV DX, LED_PORT_2
    MOV AL, 0FFH       ; 1111 1111 -> Toate LED-urile OFF
    OUT DX, AL
EXIT_OFF:
    POP DX
```

```
    RET
STINGE_LED ENDP
```

Sa zicem ca vrem sa aprindem LED-ul 9

```
MOV AL, 9          ; Selectam LED-ul 9
CALL APRINDE_LED    ; Se va aprinde (Port 0304H primeste FEH)
CALL DELAY          ; Asteptam putin
MOV AL, 9          ; Selectam portul aferent LED-ului 9
CALL STINGE_LED     ; Se va stinge (Port 0304H primeste FFH)
```

7. Rutina de afişare a unui caracter hexa pe un rang cu segmente

```
; (Anod Comun: 0 = Aprins)
;  0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
;==============================================================================
HEX_CODES  DB 0C0H, 0F9H, 0A4H, 0B0H, 99H, 92H, 82H, 0F8H
           DB 80H, 98H, 88H, 83H, 0C6H, 0A1H, 86H, 8Eh

; Adresa de baza a primului afisaj (SA1)
SEG_BASE_ADDR EQU 0308H
;==============================================================================
; RUTINA: AFISARE_CARACTER_HEXA
; Descriere: Afiseaza un caracter hexazecimal pe un anumit digit (rang).
; Input:  AL = Cifra Hexa (0 - 0FH) - ex: 0AH pentru 'A'
;         BL = Rangul/Pozitia (1 - 8)
;==============================================================================
AFISARE_CARACTER_HEXA PROC
   PUSH AX            ; Salvam registrele
   PUSH BX
   PUSH DX
   PUSH SI

   ; --- 1. Conversia Cifrei in Cod 7-Segmente ---
   LEA BX, HEX_CODES   ; Incarcam adresa tabelului
   XLAT                ; AL = HEX_CODES[AL] (Transforma cifra in cod)
   MOV SI, AX          ; Salvam codul segmentelor in SI (partea low)
```

```
    ; --- 2. Calculul Adresei Portului ---
    ; Formula: Adresa = Baza + (Rang - 1) * 4
    POP BX          ; Recuperam Rangul din stiva (era in BL)
    PUSH BX         ; Il punem la loc pentru simetrie la final

    DEC BL          ; Rang 1-8 devine 0-7
    MOV AL, BL      ; Mutam in AL pentru inmultire
    MOV CL, 4
    MUL CL          ; AL = AL * 4 (Offset-ul)

    MOV DX, SEG_BASE_ADDR ; Incarcam baza (0308H)
    ADD DX, AX      ; Adunam offset-ul la baza -> Adresa finala in DX

    ; --- 3. Afisarea pe Port ---
    MOV AX, SI      ; Recuperam codul segmentelor din SI
    OUT DX, AL      ; Trimitem codul la portul calculat

    POP SI          ; Refacem registrele
    POP DX
    POP BX
    POP AX
    RET
AFISARE_CARACTER_HEXA ENDP

;Exemplu: Afișează cifra '0' pe Rangul 1
MOV AL, 00H     ; Cifra 0
MOV BL, 1       ; Rangul 1
CALL AFISARE_CARACTER_HEXA
```

Bibliografie

1. Mircea Popa, “Sisteme cu microprocesoare”, Editura Orizonturi Universitare, 2003. (Referință principală pentru detalii despre microprocesorul 8086 și structura unui microsistem.
2. D.V. Hall, “Microprocessors and Interfacing: Programming and Hardware”, McGraw-Hill, 1992
3. 27C512 Datasheet, STMicroelectronics, 27C512 Datasheet(PDF) - STMicroelectronics
4. Datasheet-ul 8255A (Programmable Peripheral Interface), Intel, 8255A Datasheet(PDF) - Intel Corporation
5. Datasheet-ul 8251A (Programmable Communication Interface), Intel, 8251A Datasheet(PDF) - Intel Corporation
6. “LCD interfacing”, Technobyte, LCD interfacing with 8051 - 8-bit, 4-bit mode, and with 8255 PPI
7. Proiectul didactic și materialele suplimentare primite de la profesor. (Documentația de referință pentru cerințele tehnice și descrierea proiectului.